UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO
DEPARTAMENTO DE ESTATÍSTICA E INFORMÁTICA
BACHARELADO EM SISTEMAS DE INFORMAÇÃO

ALEXANDRE DE SIQUEIRA MELO JUNIOR

EFEITO DE TÉCNICAS DE NORMALIZAÇÃO SOBRE O TREINAMENTO DE REDES NEURAIS CONVOLUCIONAIS:

planejamento do repositório da pesquisa

RECIFE
2026

ALEXANDRE DE SIQUEIRA MELO JUNIOR

EFEITO DE TÉCNICAS DE NORMALIZAÇÃO SOBRE O TREINAMENTO DE REDES NEURAIS CONVOLUCIONAIS:

planejamento do repositório da pesquisa

Planejamento do repositório da pesquisa apresentado à disciplina Projetos para Sociedade II do Bacharelado em Sistemas de Informação da Universidade Federal Rural de Pernambuco, como etapa de elaboração do Trabalho de Conclusão de Curso.

Orientador: Prof. [nome do orientador]

RECIFE

2026

SUMÁRIO

1 INTRODUÇÃO 3
2 OBJETIVOS DO REPOSITÓRIO 4
3 PLATAFORMAS 5
4 ESTRUTURA DO REPOSITÓRIO 6
5 GESTÃO DOS ARTEFATOS 7
6 VERSIONAMENTO 8
6.1 RAMOS E MENSAGENS DE COMMIT 8
6.2 MARCOS DA PESQUISA 8
7 REPRODUTIBILIDADE 10
8 LICENCIAMENTO E ACESSO 11
9 CÓPIAS DE SEGURANÇA E PRESERVAÇÃO 12
10 CRONOGRAMA 13
REFERÊNCIAS 14

1 INTRODUÇÃO

Este documento apresenta o planejamento do repositório da pesquisa do Trabalho de Conclusão de Curso, que compara técnicas de normalização no treinamento de redes neurais convolucionais. O repositório reúne, num único lugar e sob controle de versões, todos os artefatos produzidos pela pesquisa: a revisão da literatura, os planejamentos, a configuração e o código do experimento, os resultados, as análises e o texto do trabalho. Ele constitui o **pacote de replicação** da pesquisa, isto é, o material que permite a outra pessoa refazer o estudo e verificar os resultados.

O repositório já foi criado e está disponível publicamente no endereço abaixo, com a estrutura descrita neste documento e os artefatos produzidos até a etapa de planejamento:

https://github.com/xandymelo/normalizacao-cnn-tcc

O arquivo README do repositório contém a versão resumida deste planejamento e é o ponto de entrada para quem acessa o material.

2 OBJETIVOS DO REPOSITÓRIO

Em pesquisas computacionais, a reprodução dos resultados depende de registrar não apenas o texto, mas também o código, os dados, a configuração e o ambiente em que os experimentos foram executados (Sandve *et al.*, 2013). Com base nessa recomendação, o repositório tem quatro objetivos:

- **reprodutibilidade** — permitir que qualquer pessoa refaça a busca bibliográfica e o experimento e obtenha os mesmos resultados;
- **rastreabilidade** — ligar cada resultado à versão exata do código, da configuração e do protocolo que o produziu;
- **transparência** — publicar o protocolo do experimento antes da coleta dos dados, para que nenhuma decisão de análise seja tomada depois de conhecidos os resultados;
- **preservação** — manter cópias seguras durante a pesquisa e arquivar a versão final com um identificador permanente.

Esses objetivos seguem os princípios FAIR, segundo os quais os dados e os artefatos de pesquisa devem ser localizáveis, acessíveis, interoperáveis e reutilizáveis (Wilkinson *et al.*, 2016). O Quadro 1 mostra como cada princípio é atendido.

Quadro 1 – Atendimento aos princípios FAIR

Princípio	Como é atendido no repositório
Localizável (*Findable*)	Endereço público no GitHub; metadados de citação no arquivo CITATION.cff; DOI permanente do Zenodo na versão final
Acessível (*Accessible*)	Repositório público, acessível por navegador ou pelo Git, sem necessidade de cadastro
Interoperável (*Interoperable*)	Dados em CSV com codificação UTF-8 e dicionário de dados; configuração em YAML; código em Python
Reutilizável (*Reusable*)	Licenças explícitas (MIT e CC BY 4.0), documentação de cada pasta e versões marcadas por tags

Fonte: elaborado pelo autor (2026).

3 PLATAFORMAS

O repositório usa três plataformas, cada uma com uma função, como descreve o Quadro 2.

Quadro 2 – Plataformas do repositório

Plataforma	Conteúdo	Justificativa
GitHub	Código, testes, configuração, notebooks, planejamentos, resultados e texto do TCC	Controle de versões com Git, histórico completo, integração contínua e acesso público
Google Drive	Resultados brutos gravados durante as execuções no Google Colab	Evita a perda de resultados quando a sessão do Colab é encerrada
Zenodo	Cópia arquivada da versão final do repositório	Preservação de longo prazo e DOI permanente para citação, com integração direta ao GitHub

Fonte: elaborado pelo autor (2026).

4 ESTRUTURA DO REPOSITÓRIO

O repositório é organizado em pastas que separam as etapas da pesquisa, como mostra o Quadro 3. Cada pasta tem um arquivo README que descreve o seu conteúdo; as pastas cujas etapas ainda não começaram descrevem o que vão conter.

Quadro 3 – Estrutura de pastas e arquivos

Pasta ou arquivo	Conteúdo	Situação
`README.md`	Apresentação da pesquisa, guia do repositório e resumo deste planejamento	Concluído
`LICENSE`, `CITATION.cff`	Licença do código e metadados de citação	Concluído
`docs/`	Planejamentos da metodologia, do experimento e do repositório, em PDF	Concluído
`revisao/`	Notebook da busca bibliográfica, triagem, fluxo da seleção, conjunto base e sua análise	Concluído para a busca piloto
`experimento/`	Configuração do experimento (`config.yaml`), programa de sorteio e ordem das 40 execuções	Concluído e testado
`src/normalizacao/`	Código do experimento: dados, modelo, treino, métricas e registro	Previsto (mês 2)
`testes/`	Testes automatizados	Parcial: testes do sorteio prontos
`notebooks/`	Notebooks de piloto, execução e análise no Google Colab	Previsto (mês 2)
`resultados/`	Arquivos `execucoes.csv` e `epocas.csv`, com dicionário de dados	Previsto (mês 3)
`analise/`	Scripts estatísticos e figuras	Previsto (mês 4)
`tcc/`	Versões do texto do TCC	Em elaboração
`.github/workflows/`	Integração contínua: execução dos testes a cada envio	Concluído

Fonte: elaborado pelo autor (2026).

5 GESTÃO DOS ARTEFATOS

Nem todo artefato da pesquisa deve ser versionado no Git. O critério adotado é versionar tudo o que é necessário para reproduzir os resultados e é pequeno o bastante para o histórico, e deixar de fora o que pode ser obtido de outra fonte, o que é protegido por direitos autorais e o que é sigiloso. O Quadro 4 resume a decisão para cada tipo de artefato.

Quadro 4 – Decisão de versionamento por artefato

Artefato	Versionado?	Tratamento
Código, testes, configuração e notebooks	Sim	Núcleo da reprodutibilidade
Planejamentos e texto do TCC	Sim	Histórico das versões entregues
Resultados da busca bibliográfica	Sim	Arquivos CSV pequenos, base da triagem
Resultados do experimento	Sim	Arquivos CSV de poucos megabytes, base da análise
Figuras finais	Sim	Usadas no texto do TCC
Conjunto Fashion-MNIST	Não	Baixado automaticamente pela biblioteca torchvision
PDFs dos artigos do conjunto base	Não	Protegidos por direitos autorais; o repositório registra apenas o DOI de cada artigo
Pesos dos modelos treinados	Não	Anexados à versão final como arquivos de *release*, fora do histórico
Chaves de API e credenciais	Nunca	Guardadas nos *Secrets* do Google Colab; excluídas pelo arquivo `.gitignore`

Fonte: elaborado pelo autor (2026).

Os arquivos de resultados seguem um dicionário de dados publicado em `resultados/README.md`, com o nome, o conteúdo e a unidade de cada coluna, conforme definido no Planejamento do Experimento.

6 VERSIONAMENTO

6.1 RAMOS E MENSAGENS DE COMMIT

O ramo principal (`main`) contém sempre uma versão que passa em todos os testes. Cada mudança é feita num ramo de curta duração, como `feat/modelo-cnn` ou `fix/medicao-tempo`, e integrada ao ramo principal por *pull request*, somente depois que a integração contínua aprova os testes.

As mensagens de commit são escritas em português, no imperativo, com um prefixo que indica o tipo de mudança: `feat` para funcionalidades, `fix` para correções, `test` para testes, `docs` para documentação, `data` para dados e resultados, `exp` para a configuração do experimento e `analise` para a análise estatística. Por exemplo: `feat: adiciona camada de Group Normalization com 8 grupos`.

6.2 MARCOS DA PESQUISA

Cada etapa da pesquisa termina com uma *tag* do Git, que fixa o estado exato do repositório naquele momento e permite recuperá-lo depois. Os marcos estão no Quadro 5.

Quadro 5 – Marcos de versão da pesquisa

Tag	Marco	Quando
`v0.1.0-planejamento`	Planejamentos da metodologia, do experimento e do repositório concluídos	Mês 1
`v0.2.0-piloto`	Código implementado e testado; experimento piloto executado	Mês 2
`v1.0.0-protocolo`	Protocolo congelado: nenhuma alteração de código, hiperparâmetro ou critério depois desta tag	Mês 2, antes das 40 execuções
`v1.1.0-resultados`	40 execuções concluídas e resultados versionados	Mês 3
`v1.2.0-analise`	Análise estatística e figuras concluídas	Mês 4
`v2.0.0-defesa`	Versão final do TCC, arquivada no Zenodo com DOI	Mês 6

Fonte: elaborado pelo autor (2026).

A tag `v1.0.0-protocolo` tem papel central: como o repositório é público, ela funciona como um registro do protocolo antes da coleta dos dados. Qualquer alteração posterior fica visível no histórico, o que dá credibilidade à afirmação de que as decisões de análise foram tomadas antes dos resultados. A primeira tag, `v0.1.0-planejamento`, foi criada com a entrega deste documento.

7 REPRODUTIBILIDADE

Além do controle de versões, o repositório adota as seguintes práticas, recomendadas por Sandve *et al.* (2013) para pesquisas computacionais:

- **configuração separada do código** — todos os hiperparâmetros, fatores, sementes e critérios ficam no arquivo `experimento/config.yaml`, e o código não contém valores fixos;
- **aleatoriedade controlada** — as sementes são registradas, e a ordem das 40 execuções é sorteada por um programa com semente fixa, cujo resultado está versionado e é conferido por um teste automatizado;
- **dependências registradas** — as bibliotecas e suas versões ficam no arquivo `requirements.txt`, com versões fixadas na tag `v1.0.0-protocolo`;
- **ambiente registrado** — cada execução grava o modelo da GPU e as versões do Python e do PyTorch junto com as métricas;
- **testes automatizados e integração contínua** — a cada envio ao GitHub, o GitHub Actions executa os testes em duas versões do Python; hoje eles verificam o sorteio da ordem das execuções e, com o código do experimento, passam a verificar as camadas de normalização, o determinismo, a detecção de divergência e a divisão dos dados;
- **notebooks sem lógica própria** — os notebooks do Google Colab apenas clonam o repositório numa *tag* fixa e chamam o código de `src/`, de modo que o que é executado é sempre uma versão identificada.

As instruções para reproduzir cada etapa estão na seção "Como reproduzir" do README.

8 LICENCIAMENTO E ACESSO

O repositório é público desde a sua criação. O código é distribuído sob a licença **MIT**, que permite o uso, a modificação e a redistribuição com a atribuição de autoria. Os textos, os dados e as figuras são distribuídos sob a licença **Creative Commons Atribuição 4.0 (CC BY 4.0)**, que permite o reuso desde que a fonte seja citada. O arquivo `CITATION.cff` contém os metadados de citação, exibidos pelo GitHub no botão *Cite this repository*.

O docente da disciplina e o orientador acompanham o repositório pelo endereço público e podem ser adicionados como colaboradores para comentar *pull requests* e *issues*.

9 CÓPIAS DE SEGURANÇA E PRESERVAÇÃO

Durante a pesquisa, são mantidas três cópias do material, em dois tipos de armazenamento, sendo uma fora do computador do autor: a cópia local, a do GitHub e a do Google Drive, que recebe os resultados das execuções no Colab assim que cada execução termina. Ao final, a versão de defesa é arquivada no Zenodo, cuja integração com o GitHub gera automaticamente um DOI permanente para a versão marcada, garantindo que o material continue acessível mesmo que o repositório do GitHub seja alterado.

10 CRONOGRAMA

O Quadro 6 relaciona as atividades no repositório aos meses do cronograma da pesquisa.

Quadro 6 – Cronograma do repositório

Mês	Atividades no repositório	Tag
1	Criação do repositório, estrutura, README, licenças, revisão da literatura e planejamentos	`v0.1.0-planejamento`
2	Código, testes e notebooks; experimento piloto; congelamento do protocolo	`v0.2.0-piloto`, `v1.0.0-protocolo`
3	Execução das 40 rodadas e versionamento dos resultados	`v1.1.0-resultados`
4	Scripts de análise e figuras	`v1.2.0-analise`
5	Texto do TCC: resultados, discussão e conclusão	—
6	Revisão final, versão de defesa e arquivamento no Zenodo	`v2.0.0-defesa`

Fonte: elaborado pelo autor (2026).

REFERÊNCIAS

SANDVE, G. K.; NEKRUTENKO, A.; TAYLOR, J.; HOVIG, E. Ten simple rules for reproducible computational research. **PLoS Computational Biology**, v. 9, n. 10, e1003285, 2013. DOI: 10.1371/journal.pcbi.1003285.

WILKINSON, M. D. *et al.* The FAIR guiding principles for scientific data management and stewardship. **Scientific Data**, v. 3, 160018, 2016. DOI: 10.1038/sdata.2016.18.